Mini-SoC RV32I in VHDL

Complete Technical Documentation

Author:

Mateus Telles Nebias

Language: VHDL (VHDL-93/2002)

Toolchain: Vivado 2025.2 | Target: Xilinx Artix-7 (xc7a35tcpg236-1)

ISA: RISC-V RV32I — Single-Cycle Implementation

Table of Contents

1.	**Project Overview**	**3**
2.	**Language, Tools and Methodology**	**4**
3.	**System Architecture**	**5**
4.	**SoC Operation — How the System Works**	**7**
5.	**The RISC-V RV32I ISA**	**10**
6.	**The Processor Datapath**	**12**
7.	**Processor Core — Module-by-Module Reference**	**13**
7.1	program_counter.vhd	13
7.2	control_unit.vhd	14
7.3	register_file.vhd	15
7.4	immediate_generator.vhd	16
7.5	alu_control.vhd	16
7.6	alu.vhd	17
7.7	branch_compare.vhd	17
7.8	load_store_unit.vhd	18
7.9	trap_unit.vhd	18
8.	**The Internal Bus — APB-like Protocol**	**20**
9.	**Memory Subsystem**	**22**
10.	**Peripherals**	**24**
10.1	gpio.vhd	24
10.2	timer.vhd	25
10.3	uart.vhd	25
10.4	dma.vhd	27
11.	**Reset Synchronisation and Clock Domains**	**29**
12.	**Verification Strategy**	**30**
13.	**Roadmap — Planned Extensions**	**32**
14.	**References**	**36**

1. Project Overview

1.1 Purpose and Goals

This document provides a thorough technical reference for the Mini-SoC RV32I, a complete system-on-chip designed from scratch in VHDL for educational and portfolio purposes. The project covers the full RTL design cycle: architectural specification, hardware implementation, functional verification through simulation, and synthesis targeting a real FPGA device.

The system is built around a RISC-V RV32I single-cycle processor core, surrounded by memory-mapped peripherals that form a minimal but fully functional SoC. Every component was specified, implemented and verified independently before integration, following a disciplined bottom-up engineering methodology.

1.2 System Capabilities

The current SoC (v1) is capable of:

- Executing the complete RV32I instruction set: arithmetic/logic operations, byte/halfword/word memory access, conditional and unconditional branches, and environment calls (ECALL, EBREAK, FENCE).
- Accessing memory-mapped peripherals using ordinary SW/LW instructions — no special I/O instructions required.
- Driving a 4-bit GPIO output and reading a digital input in real time.
- Configuring and running a 32-bit timer with comparator and interrupt generation.
- Transmitting data serially over UART (8N1) with two independent clock domains (core_clk and periph_clk), using a toggle-handshake CDC protocol.
- Executing autonomous memory-to-memory DMA transfers, offloading the CPU from per-word load/store loops.
- Detecting and handling hardware exceptions with priority: misaligned fetch, misaligned load/store, illegal instruction, ECALL, and EBREAK.

1.3 Repository Structure

```
mini-soc-rv32i/
  rtl/          VHDL synthesisable source files
  tb/           VHDL testbenches
  programs/     Compiled firmware (program.mem)
  constraints/  Artix-7 XDC pin constraints
  docs/         This document and diagrams
```

2. Language, Tools and Methodology

2.1 VHDL as a Hardware Description Language

The entire project is implemented in VHDL, standard VHDL-93/2002. VHDL (VHSIC Hardware Description Language) is a strongly-typed language used to describe the structure and behaviour of digital circuits that will be synthesised into real hardware — either FPGAs or ASICs.

Unlike conventional programming languages, VHDL is inherently concurrent. All processes execute simultaneously; within a process, signal assignments take effect only after the current simulation delta cycle completes. This reflects the physical reality of hardware, where all logic gates operate in parallel.

2.1.1 RTL Design Style

All code follows the RTL (Register Transfer Level) style, which is the standard abstraction level for synthesis tools. Key conventions adopted throughout the project:

- All registered logic uses clock-sensitive processes (clocked on the rising edge of clk).
- All combinational logic uses explicit sensitivity lists (VHDL-93/2002 compatible — process(all) is VHDL-2008 only).
- Synchronous reset on all registered elements for deterministic post-reset behaviour.
- Clean separation between combinational and sequential logic across module boundaries.

2.2 Toolchain

Tool	Purpose
Vivado 2025.2	Synthesis, implementation, bitstream generation for Artix-7
XSim	Functional simulation of all testbenches
GTKWave	Waveform viewing and analysis (.vcd)
RISC-V GCC Toolchain	Firmware compilation and program.mem generation
VS Code	RTL and testbench editing

2.3 Synthesis Target

The project targets the Xilinx Artix-7 FPGA (xc7a35tcpg236-1, Basys3 board). This device provides 33,280 6-input LUTs, 46 18Kb BRAMs (used for instruction and data memories), and 106 user I/O pins. The target core clock frequency is 100 MHz, with periph_clk at a distinct frequency to exercise real clock-domain crossing.

3. System Architecture

3.1 Top-Level Overview

The diagram below shows the complete SoC at the highest level of abstraction — all major blocks, their connections, and the two clock domains.

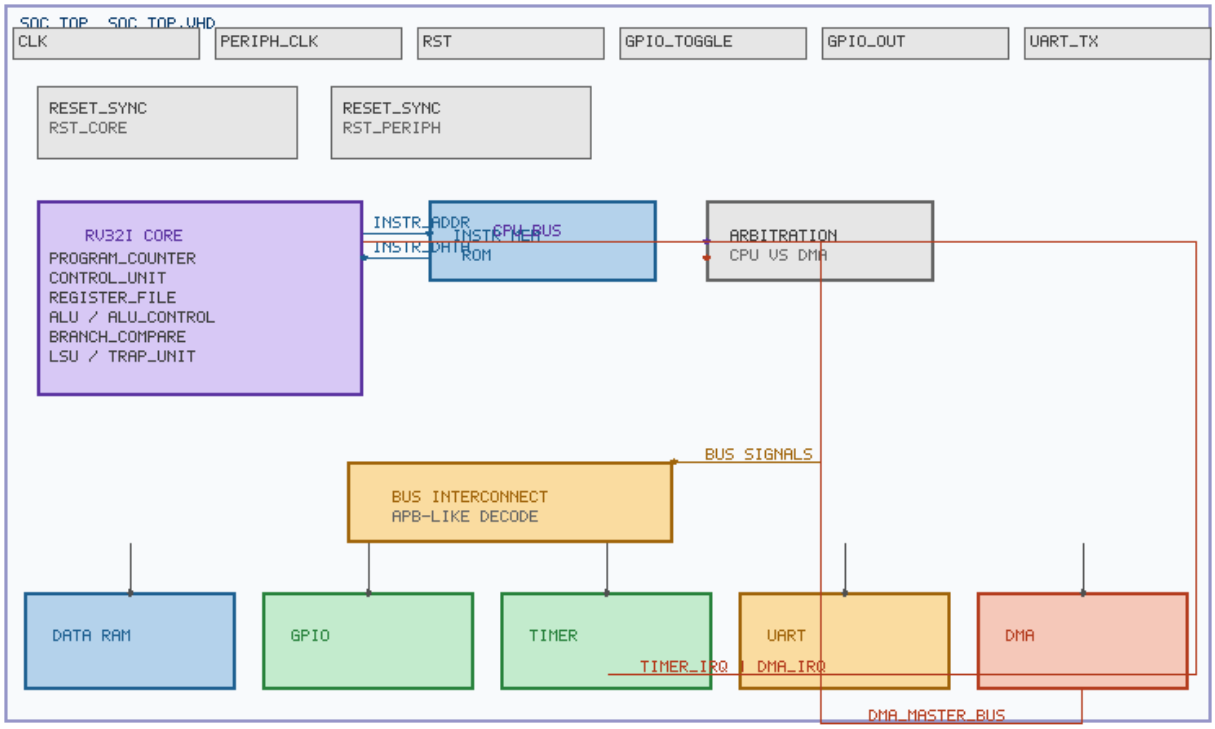

Figure 1 — soc_top.vhd: all instantiated blocks and their interconnections

3.2 Module Hierarchy

```
soc_top
  ├── reset_synchronizer  (x2 — one per clock domain)
  ├── rv32i_core          (processor core)
  │   ├── program_counter
  │   ├── control_unit
  │   ├── register_file
  │   ├── immediate_generator
  │   ├── alu_control
  │   ├── alu
  │   ├── branch_compare
  │   ├── trap_unit
  │   └── load_store_unit
```

```
├── instruction_memory  (ROM — fetch-only subsystem)
├── bus_interconnect    (APB-like router)
├── data_memory         (bus slave — data RAM)
├── gpio                (bus slave — I/O peripheral)
├── timer               (bus slave — timer + IRQ)
├── uart                (bus slave — dual-clock serial TX)
└── dma                 (bus slave for config + bus master for transfers)
```

3.3 Core vs SoC Separation

A central architectural decision is the strict separation between the processor core (rv32i_core) and the surrounding SoC (soc_top). The core exposes two clean interfaces to the SoC and nothing more:

- Instruction fetch interface: instr_addr (output) and instr_rdata (input). Connected directly to instruction_memory outside the core.
- Data bus interface (APB-like): bus_addr, bus_wdata, bus_wstrb, bus_write, bus_read, bus_valid (outputs) and bus_rdata, bus_ready, bus_error (inputs). Connected to the bus_interconnect which routes to the appropriate peripheral.

This clean boundary means the core can be reused in any SoC context without modification. The core has no knowledge of the system topology around it.

3.4 Clock Domains

Domain	Modules	Purpose
`core_clk`	rv32i_core, bus_interconnect, data_memory, gpio, timer, dma	Main system clock. All control logic and memory accesses.
`periph_clk`	UART TX engine (internal to uart.vhd)	Serial peripheral clock. Independent frequency enables flexible baud rates.

4. SoC Operation — How the System Works

This chapter explains how the SoC operates end-to-end, from power-on reset to firmware execution and peripheral interaction. Understanding these flows is essential to appreciate how the individual modules described in subsequent chapters work together as a coherent system.

4.1 Power-On and Reset Sequence

When the FPGA is powered on or the rst signal is asserted, the following sequence occurs:

1. rst is asserted high. Both reset_synchronizer instances immediately assert their outputs (rst_core = '1', rst_periph = '1') asynchronously — no clock edge needed.
2. All flip-flops in the system enter their reset state: the program counter is forced to 0x00000000, all DMA state machines enter IDLE, timer counters clear, and GPIO outputs are zeroed.
3. When rst is released (deasserted), each domain's reset_synchronizer allows the '0' to propagate through two flip-flops before releasing rst_core and rst_periph. This guarantees both domains exit reset synchronously with their respective clocks, preventing metastability.
4. The core begins executing from address 0x00000000 — the first word of instruction_memory.

4.2 Instruction Fetch and Execution Loop

Every clock cycle, the processor executes one complete instruction. The sequence within each cycle is fully combinational (no intermediate registers):

5. The program counter (PC) drives instr_addr. The instruction_memory immediately returns the 32-bit instruction on instr_rdata — combinationally, within the same cycle.
6. The 32-bit instruction is simultaneously decoded by control_unit (which generates all datapath control signals), immediate_generator (which extracts the sign-extended immediate), and the register file (which reads rs1 and rs2 combinationally).
7. The ALU receives operands (rs1 or PC for operand A; rs2 or immediate for operand B) selected by multiplexers controlled by control_unit. The ALU computes the result.
8. If the instruction is a load or store, the load_store_unit generates byte-enables and aligned write data (for stores), or extracts and sign-extends the loaded value (for loads), using the ALU result as the memory address.
9. The bus_valid, bus_read, bus_write signals are driven. The bus_interconnect decodes the address and routes the access to the correct peripheral, which responds combinationally.
10. The trap_unit evaluates all exception conditions simultaneously. If a trap is detected (trap_taken = '1'), the side effects (register write, memory access) are suppressed.
11. On the rising clock edge: the PC register captures next_pc (PC+4, branch target, JAL/JALR target, or trap vector depending on priority), and the register file captures the writeback value if reg_write_g = '1'.

4.3 Memory-Mapped Peripheral Access

From the CPU's perspective, reading or writing a peripheral is identical to a regular memory access. For example, to read the current timer count, the firmware executes:

```
li  t0, 0x20        # TIMER_COUNT address
lw  t1, 0(t0)       # loads the 32-bit counter value into t1
```

What happens at the hardware level:

12. The LW instruction causes the LSU to drive bus_addr = 0x00000020, bus_read = '1', bus_valid = '1'.
13. In soc_top, the arbitration mux passes the CPU signals to bus_interconnect (since dma_active = '0').
14. bus_interconnect decodes address 0x20: this matches the TIMER range. It asserts timer_sel = '1' and passes addr, read, valid to the timer.
15. The timer peripheral sees sel = '1', valid = '1', read_en = '1'. It drives timer_rdata = TIMER_COUNT register, timer_ready = '1'.
16. bus_interconnect muxes timer_rdata back to fabric_rdata. The core receives bus_rdata = TIMER_COUNT value.
17. The LSU extracts the full 32-bit word (LW, no sign extension needed) and writes it to register t1.

4.4 Interrupt Handling

The SoC currently generates two IRQ sources: timer_irq (when TIMER_COUNT matches TIMER_CMP and irq_enable is set) and dma_irq (when a DMA transfer completes and irq_enable is set). These are OR'd in soc_top to form irq_combined, which drives the irq_in port of rv32i_core.

In the current implementation, irq_in is connected to the core interface but the core does not yet act on external interrupts — full interrupt handling is planned as the next major extension (see Chapter 12). The trap_unit handles synchronous exceptions (illegal instruction, misaligned access, ECALL, EBREAK) and redirects the PC to the trap vector at address 0x00000080.

4.5 DMA Transfer Flow

A DMA transfer proceeds as follows from the firmware's perspective:

18. CPU writes DMA_SRC_ADDR, DMA_DST_ADDR, DMA_LENGTH (in words), and DMA_CTRL[1] (irq_enable) via store instructions.
19. CPU writes DMA_CTRL[0] = 1 (start pulse). The DMA validates the configuration. If valid, it asserts active_o = '1'.
20. In soc_top, the arbitration mux switches to the DMA as bus master. The CPU is disconnected from the fabric response (cpu_bus_rdata = 0, cpu_bus_ready = '0').
21. The DMA's FSM cycles through READ_REQ and WRITE_REQ states, reading one word from the source address and writing it to the destination address each iteration, incrementing both pointers and decrementing the counter.

22. When the counter reaches zero, the DMA enters DONE state: sets done = '1' in DMA_STATUS, pulses irq if enabled, and releases active_o = '0'.
23. The CPU regains bus mastership. Firmware polls DMA_STATUS[0] (busy) or waits for the interrupt before continuing.

5. The RISC-V RV32I ISA

5.1 What is RISC-V

RISC-V is an open-standard Instruction Set Architecture (ISA) originally developed at UC Berkeley in 2010. Unlike proprietary ISAs (x86, ARM), RISC-V is royalty-free and can be implemented by anyone. The RISC (Reduced Instruction Set Computer) philosophy emphasises a small, regular instruction set with fixed-width 32-bit instructions (in the base RV32I variant), load/store memory access model, and only register-to-register arithmetic.

5.2 RV32I Instruction Formats

All RV32I instructions are exactly 32 bits wide. The ISA defines six encoding formats:

Format	Used by	Immediate field layout
R-type	ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU	No immediate — uses funct7[31:25] and funct3[14:12]
I-type	ADDI, loads, JALR	imm[11:0] from bits [31:20], sign-extended to 32 bits
S-type	SB, SH, SW	imm[11:5] from bits [31:25], imm[4:0] from bits [11:7]
B-type	BEQ, BNE, BLT, BGE, BLTU, BGEU	Scattered bits; 13-bit signed, multiples of 2
U-type	LUI, AUIPC	imm[31:12] from bits [31:12]; lower 12 bits zeroed
J-type	JAL	Scattered bits; 21-bit signed, multiples of 2

5.3 Single-Cycle Implementation

In a single-cycle processor, every instruction — regardless of type or complexity — completes in exactly one clock cycle. There are no intermediate registers between functional units: fetch, decode, execute, memory access, and writeback are all part of a single combinational cloud that resolves between two clock edges. The longest delay path through this combinational cloud — the critical path — determines the maximum operating frequency.

The primary advantage is radical simplicity: there are no data hazards, no control hazards, no forwarding networks, no stall detection logic. The trade-off is a lower maximum clock frequency, because the clock period must be long enough for the critical path — the slowest possible combination of instruction memory read + decode + ALU + data bus access + writeback mux — to fully settle. A pipelined design breaks this chain into shorter segments separated by registers, allowing a much faster clock at the cost of significantly higher control complexity.

5.3.1 Single-Cycle vs 5-Stage Pipeline

Aspect	Single-Cycle (this project) — no pipeline	5-Stage Pipeline
CPI	Always 1	~1 ideal, >1 with hazards
Clock frequency	Limited by full critical path	Limited by slowest stage (~5x faster clock)
Control complexity	Low — no hazards, no forwarding	High — data hazards, control hazards, stalls

6. The Processor Datapath — Single-Cycle Combinational Flow

The datapath is the set of physical paths through which data flows during instruction execution. In a single-cycle design, the ENTIRE datapath from instruction fetch to register writeback is a single continuous combinational circuit — there are no pipeline registers, no intermediate flip-flops, no latches between steps. Data propagates as electrical signals through logic gates in one uninterrupted flow.

The five 'phases' described below are purely conceptual divisions of this combinational path — like labelling sections of a wire. They are NOT pipeline stages. There is no clock edge between them, no parallelism, and no latency beyond propagation delay. Only two state elements in the entire design capture values at the clock edge: the Program Counter register and the Register File write port.

> **Key distinction:** In a 5-stage pipeline, 'Stage 3 — Execute' means a flip-flop boundary exists before and after the ALU, and multiple instructions occupy different stages simultaneously. In our single-cycle design, the ALU simply sits on a wire between the register file output and the bus address input. The word 'phase' is used below instead of 'stage' to avoid this confusion.

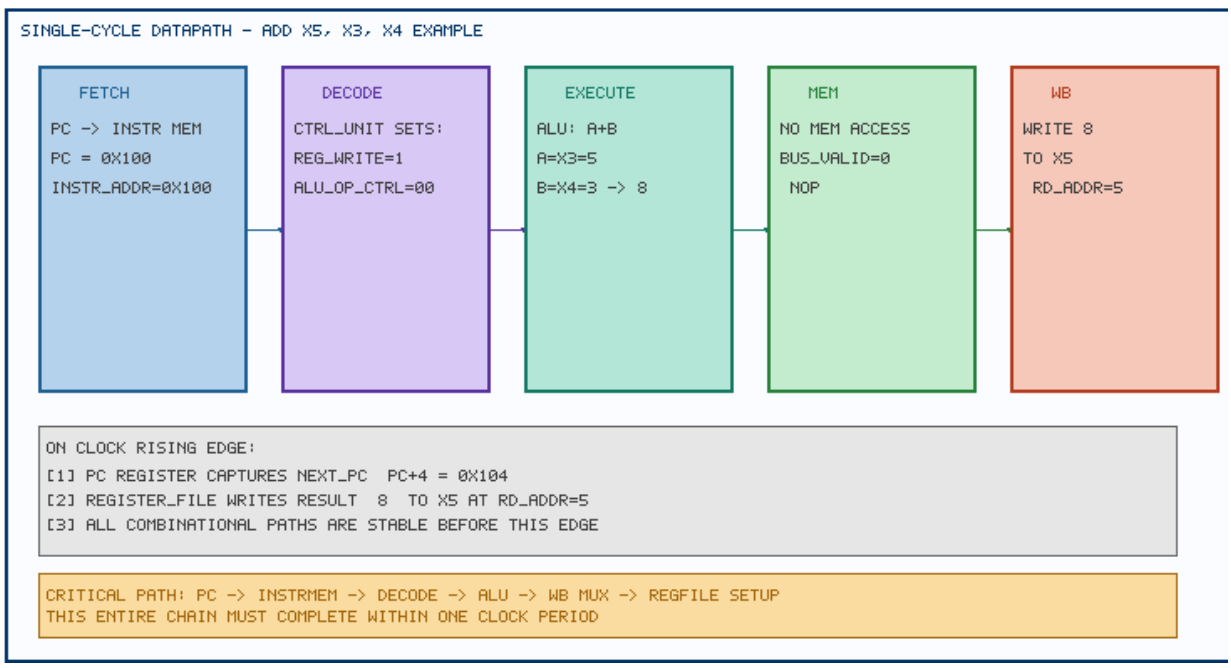

Figure 2 — Single-cycle datapath illustrated with an ADD x5, x3, x4 example

6.1 Phase 1 — Instruction Fetch

The Program Counter drives instr_addr continuously. instruction_memory responds combinationally with the 32-bit instruction. The next PC value (next_pc) is computed combinationally with the following priority:

24. Trap: if trap_taken = '1', next_pc = trap_target (fixed vector at 0x00000080).
25. JALR: if is_jalr = '1', next_pc = (rs1 + imm) AND 0xFFFFFFFE (bit 0 cleared per spec).
26. JAL: if is_jal = '1', next_pc = PC + imm.
27. Branch taken: if is_branch = '1' AND branch_taken = '1', next_pc = PC + imm.
28. Default: next_pc = PC + 4.

6.2 Phase 2 — Decode

The 32-bit instruction is decoded simultaneously by three modules: control_unit generates all datapath control signals from the opcode; immediate_generator extracts and sign-extends the immediate field; register_file reads rs1 and rs2 combinationally.

6.3 Phase 3 — Execute

Two multiplexers select ALU operands: operand A is either PC (for AUIPC) or rs1; operand B is either the immediate (for I/S/B/U/J-type) or rs2. The ALU computes the result and drives alu_result. Simultaneously, branch_compare evaluates the branch condition directly on raw rs1/rs2 values, avoiding any ALU dependency.

6.4 Phase 4 — Memory / Bus Access

The load_store_unit receives alu_result as the memory address. For stores, it generates aligned write data and byte-enables. For loads, it extracts the correct bytes from the bus response and applies sign or zero extension. The bus signals are driven and the bus_interconnect routes the access to the correct peripheral.

6.5 Phase 5 — Writeback

A final multiplexer (wb_sel) selects the value written back to the register file:

- 00 → ALU result (arithmetic/logic, LUI, AUIPC)
- 01 → Load result (load instructions)
- 10 → PC + 4 (JAL, JALR — saves return address)

The write is gated by reg_write_g = reg_write AND (NOT trap_taken), ensuring no register is corrupted during exceptions.

7. Processor Core — Module-by-Module Reference

The diagram below shows the rv32i_core internal structure — all sub-modules and their signal connections.

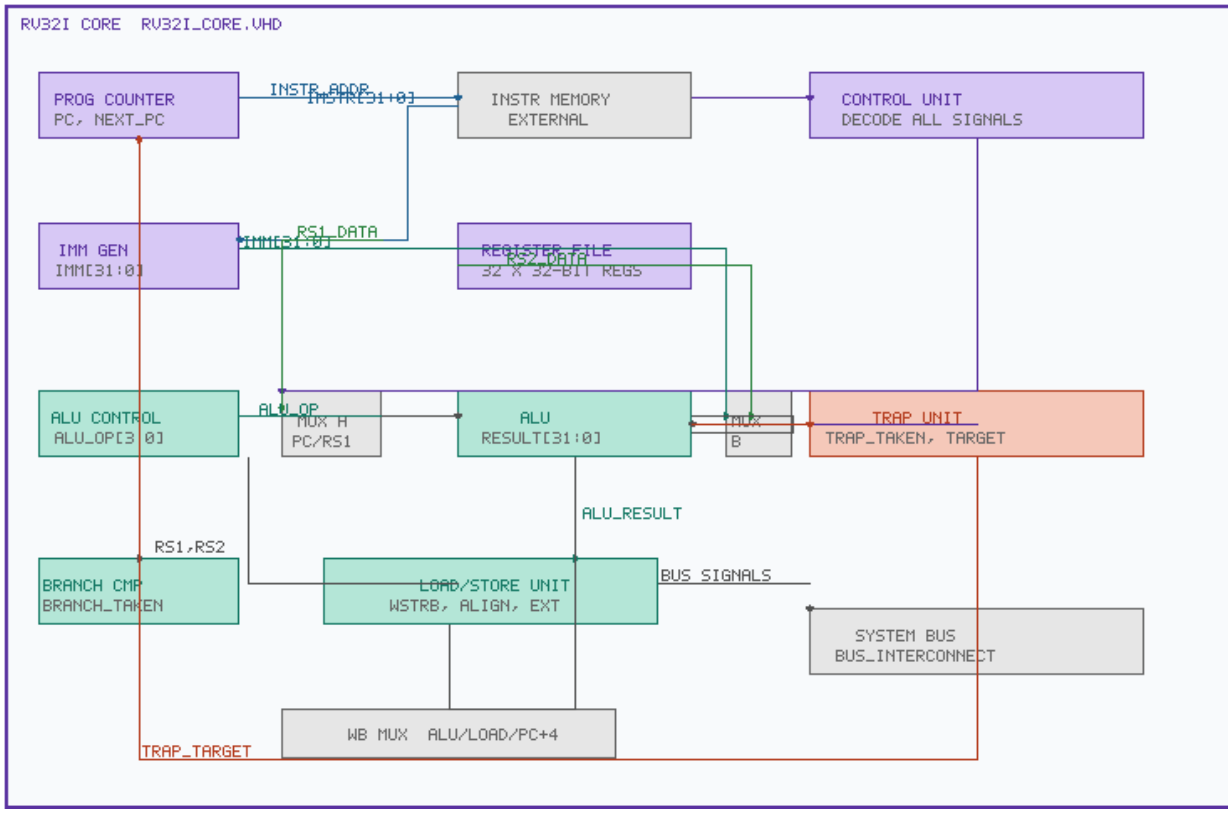

Figure 3 — rv32i_core internal structure with all sub-module interfaces

7.1 program_counter.vhd

7.1.1 Role within the Core

Holds the address of the current instruction and advances it every clock cycle. It is the only state register that directly controls the instruction stream. Its output (pc) drives the instruction_memory address input continuously; next_pc is computed by the surrounding logic in rv32i_core and fed back.

7.1.2 Interface

Signal	Dir / Type	Description
clk	in std_logic	System clock
rst	in std_logic	Synchronous reset — forces PC to 0x00000000

Signal	Dir / Type	Description
enable	in std_logic	Update enable — held at '1'; reserved for future stall support
next_pc [31:0]	in std_logic_vector	Next address computed by datapath
pc [31:0]	out std_logic_vector	Current PC — drives instruction_memory address

7.2 control_unit.vhd

7.2.1 Role within the Core

Fully combinational decoder. Receives the 32-bit instruction and generates every control signal needed by the datapath, LSU, and trap unit. It has no state — the same instruction always produces the same set of control signals.

7.2.2 Interface

Signal	Dir / Type	Description
instr [31:0]	in std_logic_vector	Full 32-bit instruction
reg_write	out std_logic	'1' enables write to register file
alu_src_a_pc	out std_logic	'1' selects PC as ALU operand A (AUIPC)
alu_src_b_imm	out std_logic	'1' selects immediate as ALU operand B
alu_op_ctrl [1:0]	out std_logic_vector	Operation class code fed to alu_control
is_branch / is_jal / is_jalr	out std_logic	PC control intent signals
wb_sel [1:0]	out std_logic_vector	Writeback source select: 00=ALU, 01=Load, 10=PC+4
mem_re / mem_we	out std_logic	Read/write enable for memory access
mem_size [1:0]	out std_logic_vector	Access granularity: 00=byte, 01=halfword, 10=word
mem_unsigned	out std_logic	'1' for zero-extending loads (LBU, LHU)
ecall / ebreak / illegal_insn	out std_logic	Exception source flags

7.2.3 Opcode Decode Table

Opcode	Format	Instructions
`0110011`	R-type	ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU

Opcode	Format	Instructions
0010011	I-type (ALU)	ADDI, ANDI, ORI, XORI, SLTI, SLTIU, SLLI, SRLI, SRAI
0000011	I-type (Load)	LB, LH, LW, LBU, LHU
0100011	S-type	SB, SH, SW
1100011	B-type	BEQ, BNE, BLT, BGE, BLTU, BGEU
0110111	U-type	LUI
0010111	U-type	AUIPC
1101111	J-type	JAL
1100111	I-type (Jump)	JALR
1110011	I-type (Env)	ECALL, EBREAK, FENCE (NOP)

7.3 register_file.vhd

7.3.1 Role within the Core

Implements the 32 general-purpose registers (x0–x31), each 32 bits wide. Reads are fully combinational (rs1_data and rs2_data are available immediately after rs1_addr/rs2_addr change). Write is synchronous on the rising clock edge. Register x0 is hardwired to zero — writes are ignored and reads always return 0x00000000.

7.3.2 Interface

Signal	Dir / Type	Description
clk	in std_logic	Clock
we	in std_logic	Write enable — active high, synchronous
rs1_addr [4:0]	in std_logic_vector	First read port address
rs2_addr [4:0]	in std_logic_vector	Second read port address
rd_addr [4:0]	in std_logic_vector	Write port address
rd_data_in [31:0]	in std_logic_vector	Data to write (from wb_data)
rs1_data_out [31:0]	out std_logic_vector	Combinational read of rs1
rs2_data_out [31:0]	out std_logic_vector	Combinational read of rs2

7.4 immediate_generator.vhd

7.4.1 Role within the Core

Extracts the immediate field from the instruction and sign-extends it to 32 bits, according to the instruction format. Fully combinational. The format is determined implicitly from the opcode — the module handles all five RV32I immediate encodings.

7.4.2 Immediate Formats

Format	Source bits	Usage
I	[31:20]	Loads, ALU-immediate, JALR — 12-bit signed
S	[31:25] + [11:7]	Stores — 12-bit signed
B	scattered	Branches — 13-bit signed, must be multiple of 2
U	[31:12]	LUI, AUIPC — 20 bits, concatenated with 12 zeros
J	scattered	JAL — 21-bit signed, must be multiple of 2

7.5 alu_control.vhd

7.5.1 Role within the Core

Translates the 2-bit alu_op_ctrl (instruction class from control_unit) plus funct3 and funct7 fields into a 4-bit ALU operation code. Fully combinational.

7.5.2 Operation Code Table

4-bit code	Operation	Instructions
`0000`	ADD	ADD, ADDI, JALR, loads, stores, AUIPC
`0001`	SUB	SUB
`0010`	LUI (pass-B)	LUI — passes immediate directly to result
`0011`	SLL	SLL, SLLI
`0100`	SLT (signed)	SLT, SLTI
`0101`	SLTU (unsigned)	SLTU, SLTIU
`0110`	XOR	XOR, XORI
`0111`	SRL	SRL, SRLI
`1000`	SRA	SRA, SRAI
`1001`	OR	OR, ORI
`1010`	AND	AND, ANDI

7.6 alu.vhd

7.6.1 Role within the Core

Executes the arithmetic or logical operation on two 32-bit operands. Fully combinational. The VHDL implementation uses IEEE.NUMERIC_STD signed and unsigned types to correctly synthesise signed operations (ADD, SUB, SLT, SRA) and unsigned comparisons (SLTU, BLTU, BGEU).

7.6.2 Interface

Signal	Dir / Type	Description
A [31:0]	in std_logic_vector	Operand A (rs1 or PC)
B [31:0]	in std_logic_vector	Operand B (rs2 or immediate)
alu_op [3:0]	in std_logic_vector	4-bit operation code from alu_control
result [31:0]	out std_logic_vector	32-bit result
zero	out std_logic	'1' when result = 0 (available for external use)

7.7 branch_compare.vhd

7.7.1 Role within the Core

Evaluates branch conditions directly on raw rs1 and rs2 values — independently of the ALU. This design choice avoids the need to configure the ALU specifically for branch evaluation and simplifies the control logic.

7.7.2 Supported Conditions

funct3	Instruction	Condition evaluated
000	BEQ	rs1 = rs2
001	BNE	rs1 != rs2
100	BLT	rs1 < rs2 (signed)
101	BGE	rs1 >= rs2 (signed)
110	BLTU	rs1 < rs2 (unsigned)
111	BGEU	rs1 >= rs2 (unsigned)

7.8 load_store_unit.vhd

7.8.1 Role within the Core

The LSU bridges the ISA memory model (byte-addressable, byte/halfword/word granularity) and the internal bus (word-addressed with byte strobes). It is the most corner-case-rich module in the datapath.

7.8.2 Responsibilities

- Store path: generates byte-enables (wstrb) and aligns the store data within the 32-bit bus word according to addr[1:0] and access size.
- Load path: extracts the correct bytes from the 32-bit bus response and applies sign (LB, LH) or zero (LBU, LHU) extension.
- Misalignment detection: LH/SH require addr[0] = 0; LW/SW require addr[1:0] = 00.

Example — LB x5, 2(x3): If x3=0x100, addr=0x102. addr[1:0]=10. Bus reads word at 0x100 = 0xAABBCCDD. LSU extracts bits[23:16] = 0xCC. Sign-extend: 0xCC has bit7=1, result = 0xFFFFFFCC.

7.9 trap_unit.vhd

7.9.1 Role within the Core

Detects and prioritises hardware exceptions, asserting trap_taken and providing trap_target (the exception vector address). The current implementation uses a fixed trap vector (generic TRAP_VECTOR, default 0x00000080) without CSR registers.

7.9.2 Exception Priority

Priority	Source	trap_cause	Description
1 (highest)	misaligned_fetch	0001	PC[1:0] != 00 — misaligned instruction fetch
2	misaligned_ls	0010	Load/store with misaligned address
3	illegal_insn	0011	Unrecognised opcode
4	ecall	0100	ECALL instruction executed
5 (lowest)	ebreak	0101	EBREAK instruction executed

7.9.3 Effect of a Trap

- The PC captures trap_target on the next rising edge — redirecting execution to the handler.
- reg_write_g is forced to '0' — no register is corrupted.
- mem_re_g and mem_we_g are forced to '0' — no spurious memory access occurs.

8. The Internal Bus — APB-like Protocol

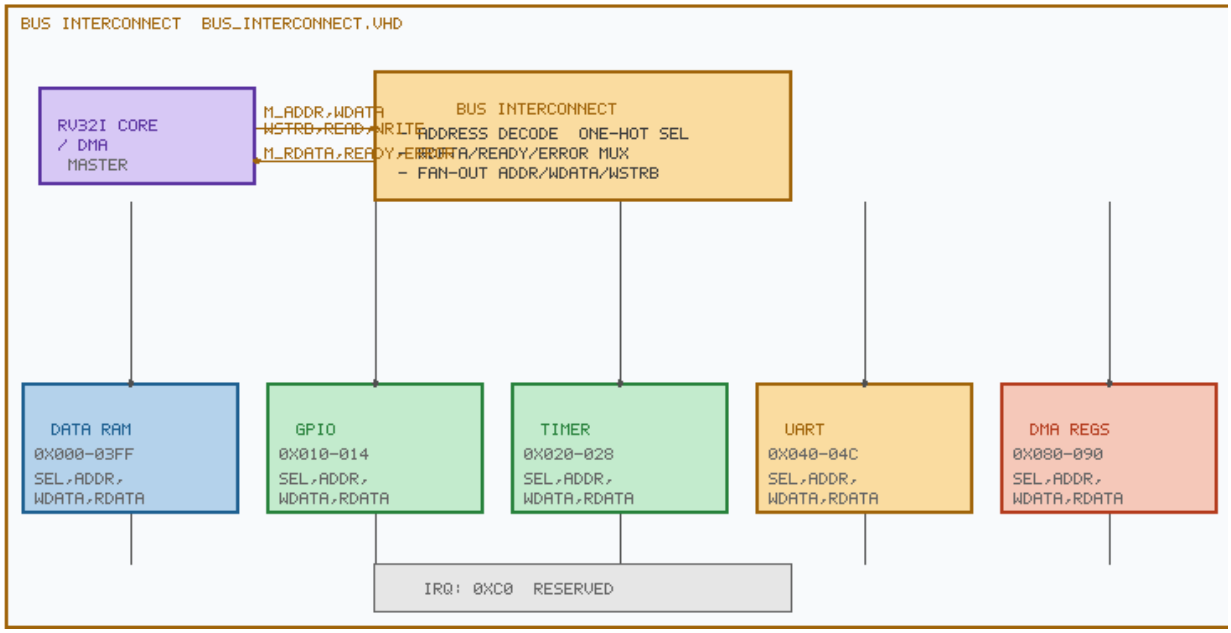

Figure 4 — bus_interconnect.vhd: address decode, fan-out, and response multiplexing

8.1 Design Philosophy

The internal bus is an APB-like simplified protocol — inspired by ARM's AMBA APB (Advanced Peripheral Bus) but without the full specification complexity. The design criteria were: simple enough to implement without engineering overhead, formal enough to document and reuse across all peripherals, and extensible to support wait-states, multiple masters, and bus errors without interface redesign.

8.2 Bus Signals

8.2.1 Master Interface (CPU / DMA → Bus)

Signal	Dir / Type	Meaning
bus_addr [31:0]	out	Byte-addressed access address
bus_wdata [31:0]	out	Write data
bus_wstrb [3:0]	out	Byte enables: bit N enables byte N
bus_write	out	'1' for write transaction
bus_read	out	'1' for read transaction
bus_valid	out	'1' when a transaction is active (= write OR read)

Signal	Dir / Type	Meaning
bus_rdata [31:0]	in	Read data from selected slave
bus_ready	in	'1' when transaction is complete
bus_error	in	'1' on address decode error or slave error

8.2.2 Slave Interface (Bus → Peripheral)

Signal	Dir	Meaning
sel	in	This slave is selected — generated by bus_interconnect
addr, wdata, wstrb	in	Forwarded from master bus
write_en, read_en, valid	in	Forwarded from master bus
rdata [31:0]	out	Read data from this slave
ready	out	'1' when this slave has a response ready
error	out	'1' on local error (e.g. invalid register offset)

8.3 Protocol Rules

- bus_write and bus_read are mutually exclusive — never both '1' simultaneously.
- bus_valid is the logical OR of bus_write and bus_read.
- A slave must only respond when sel = '1' AND valid = '1'.
- All current slaves respond in 1 cycle (ready = '1' immediately). The ready signal is present from the start to allow future wait-state introduction without interface change.
- Access to an unmapped address: bus_interconnect asserts error = '1', rdata = 0, ready = '1'.

8.4 CPU/DMA Arbitration

The soc_top implements a simple 2-master arbitration policy (takeover). When the DMA is active (dma_active = '1'), it takes exclusive ownership of the bus. The CPU's signals are disconnected from the fabric response:

```
bus_addr  <= dma_m_addr  when dma_active = '1' else cpu_bus_addr;
cpu_bus_rdata <= fabric_rdata when dma_active = '0' else (others => '0');
```

Design note: This is an intentional simplification. Firmware must wait for DMA completion (polling DMA_STATUS[0] or using the DMA IRQ) before issuing data accesses. Full round-robin arbitration is planned as a future enhancement.

9. Memory Subsystem

9.1 instruction_memory.vhd

A read-only memory (ROM) implemented as an array of 32-bit words. Content is loaded at elaboration time from program.mem using VHDL's hread function. Reads are fully combinational — the address drives instr_rdata without any registered stage. In synthesis, the content is embedded into the bitstream as an initialised BRAM.

9.2 data_memory.vhd

The data RAM implements 256 words of 32 bits (1 KiB). It supports byte-enable writes (wstrb), allowing the LSU to write individual bytes without affecting adjacent bytes in the same word. Reads are combinational; writes are synchronous on the rising clock edge. The interface follows the standard bus slave protocol.

9.3 Complete Memory Map

Address Range	Register	Access	Description
0x00000000 – 0x000003FF	Data RAM	RW	256 x 32-bit words — main data memory
0x00000010	GPIO_OUT	RW	GPIO output bits [3:0]
0x00000014	GPIO_IN	RO	GPIO input bit [0] (gpio_toggle pin)
0x00000020	TIMER_COUNT	RO	Current timer counter value
0x00000024	TIMER_CMP	RW	Comparator value — triggers IRQ on match
0x00000028	TIMER_CTRL	RW	bit0=enable, bit1=irq_enable, bit2=clear
0x00000040	UART_TXDATA	WO	Byte to transmit (bits [7:0] used)
0x00000044	UART_STATUS	RO	bit0=tx_ready, bit1=tx_busy
0x00000048	UART_CTRL	RW	bit0=enable
0x0000004C	UART_BAUDDIV	RW	Baud rate divisor
0x00000080	DMA_SRC_ADDR	RW	DMA source address
0x00000084	DMA_DST_ADDR	RW	DMA destination address
0x00000088	DMA_LENGTH	RW	Number of 32-bit words to transfer
0x0000008C	DMA_CTRL	RW	bit0=start (pulse), bit1=irq_enable
0x00000090	DMA_STATUS	RO	bit0=busy, bit1=done, bit2=error

Address Range	Register	Access	Description
0x000000C0 – 0xCC	IRQ Controller	—	Reserved for future interrupt controller
0x00000100 – 0x01FF	Expansion	—	Reserved for debug regs, perf counters

Architectural note: Addresses 0x10–0x28 fall within the RAM address window (0x000–0x3FF) but are intercepted by the bus_interconnect before reaching data_memory. From the software perspective, they are MMIO registers, not RAM.

10. Peripherals

10.1 gpio.vhd

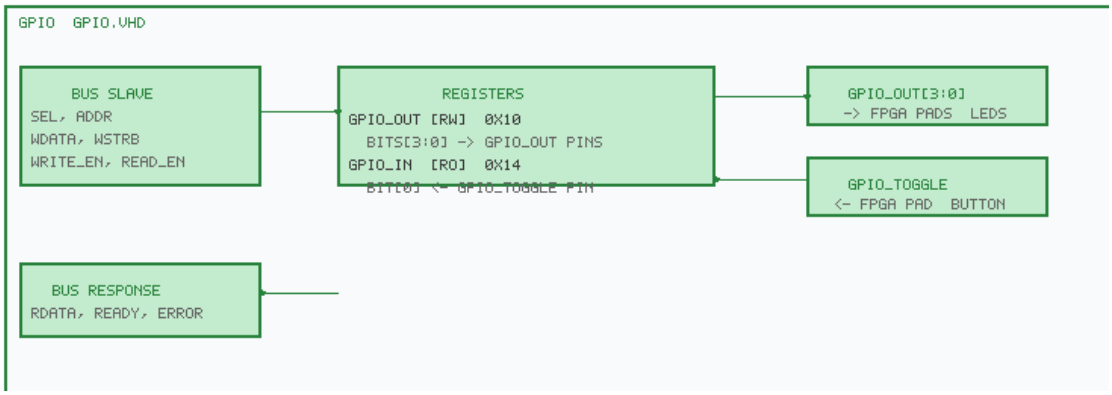

Figure 5 — gpio.vhd: bus slave interface and external pin connections

10.1.1 Role in the SoC

Provides a 4-bit digital output (gpio_out) driving FPGA pads (e.g. LEDs on Basys3) and a 1-bit digital input (gpio_toggle) reading an external signal (e.g. a push button). Accessible by firmware via standard load/store instructions.

10.1.2 Register Map

- GPIO_OUT (0x0010, RW): bits [3:0] control gpio_out[3:0]. Synchronous write, combinational read.
- GPIO_IN (0x0014, RO): bit [0] reflects the current state of the gpio_toggle input pin. Combinational read — no latency.

10.1.3 Connections to Adjacent Modules

- ← bus_interconnect: sel, addr, wdata, wstrb, write_en, read_en, valid
- → bus_interconnect: rdata, ready, error
- → soc_top / FPGA pad: gpio_out[3:0]
- ← soc_top / FPGA pad: gpio_toggle

10.2 timer.vhd

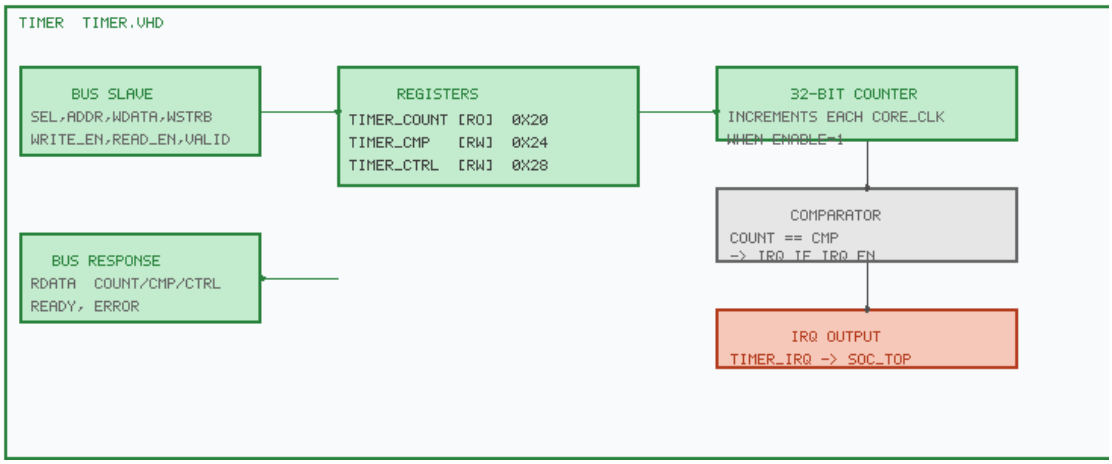

Figure 6 — timer.vhd: counter, comparator, and IRQ generation

10.2.1 Role in the SoC

A 32-bit free-running counter with comparator and interrupt output. Used by firmware for precise delays, periodic event generation, and interrupt-driven timing.

10.2.2 Operation

When TIMER_CTRL[0] (enable) = '1', TIMER_COUNT increments by 1 every core_clk cycle. When TIMER_COUNT reaches TIMER_CMP, the irq signal is asserted for one cycle if TIMER_CTRL[1] (irq_enable) = '1'. Writing '1' to TIMER_CTRL[2] (clear) resets TIMER_COUNT to zero.

10.2.3 Connections to Adjacent Modules

- ← bus_interconnect: sel, addr, wdata, wstrb, write_en, read_en, valid
- → bus_interconnect: rdata (COUNT/CMP/CTRL values), ready, error
- → soc_top: timer_irq (combined with dma_irq → rv32i_core irq_in)

10.3 uart.vhd

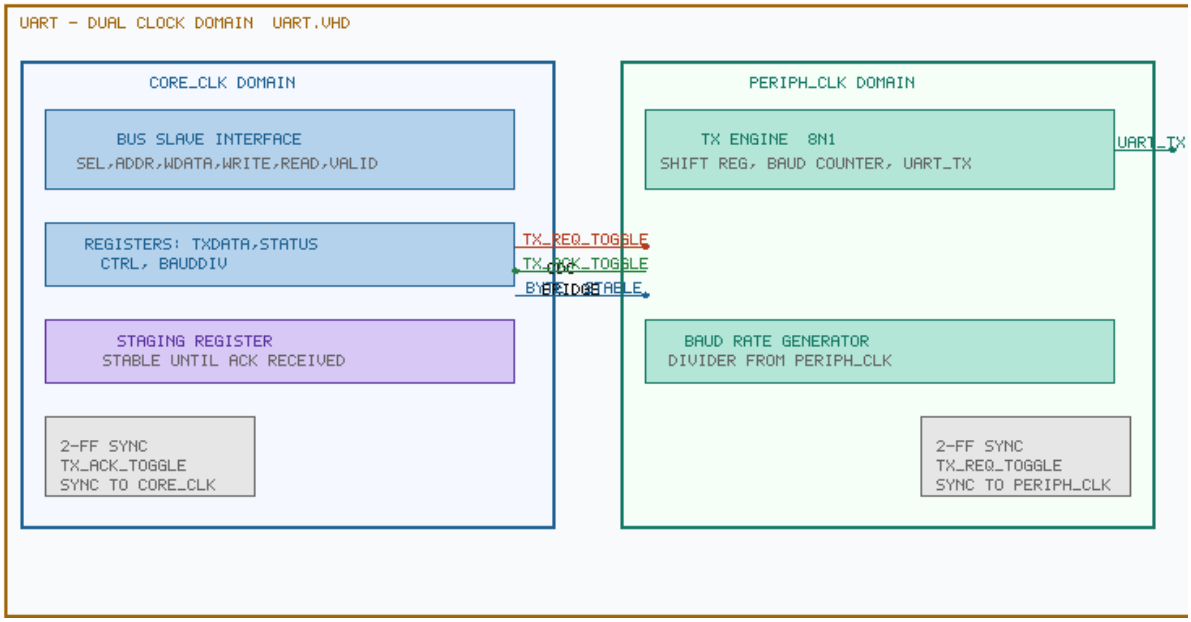

Figure 7 — uart.vhd: dual-clock domain structure and CDC handshake

10.3.1 Role in the SoC

Implements serial transmission in 8N1 format (8 data bits, no parity, 1 stop bit) — the most widely supported asynchronous serial configuration. The UART is the primary debug and demo interface of the SoC. It operates across two clock domains, making it the most architecturally complex peripheral.

10.3.2 Dual-Clock Architecture

The module is logically split into two parts that live in different clock domains:

- core_clk domain: bus slave interface, configuration registers (TXDATA, STATUS, CTRL, BAUDDIV), staging register holding the byte to transmit, and the CDC request flip-flop.
- periph_clk domain: baud rate counter, shift register, start/stop bit generation, and the physical uart_tx output.

10.3.3 CDC Toggle Handshake Protocol

Crossing multi-bit data between asynchronous domains safely without a FIFO is achieved through a toggle handshake:

29. Firmware writes a byte to UART_TXDATA. The byte is stored in the staging register (core_clk domain) and will remain stable until the transfer is acknowledged.
30. tx_req_toggle is toggled in core_clk domain. The UART enters tx_busy state.
31. In periph_clk domain, a 2-FF synchroniser detects the toggle (edge detection on synchronised value).
32. The TX engine captures the stable byte from the staging register and begins the 8N1 serial transmission.
33. Upon accepting the byte, tx_ack_toggle is toggled in periph_clk domain.

34. Back in core_clk domain, a 2-FF synchroniser detects the ack toggle. tx_ready is reasserted.

10.3.4 Connections to Adjacent Modules

- ← bus_interconnect: sel, addr, wdata, wstrb, write_en, read_en, valid
- → bus_interconnect: rdata, ready, error
- ← soc_top: core_clk, core_rst, periph_clk, periph_rst
- → soc_top / FPGA pad: uart_tx

10.4 dma.vhd

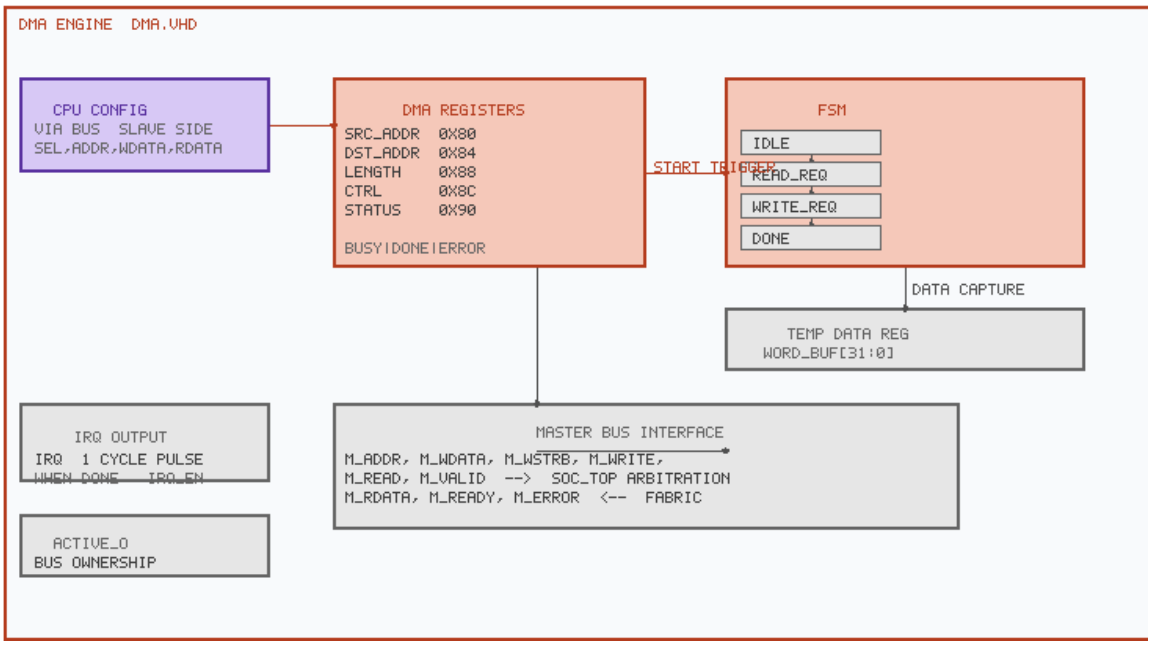

Figure 8 — dma.vhd: slave configuration interface, FSM, and master bus interface

10.4.1 Role in the SoC

A Direct Memory Access controller that moves data between memory regions autonomously, without the CPU executing individual load/store instructions. This is a fundamental SoC capability demonstrating bus master arbitration and autonomous data movement.

10.4.2 Scope

- Word-based (32-bit) transfers: RAM to RAM.
- Single channel — one active transfer at a time.
- Same clock domain as the rest of the system (core_clk).
- Bus takeover during transfer — no full round-robin arbitration.
- 1-cycle IRQ pulse on completion when irq_enable = '1'.

10.4.3 Internal FSM

State	Behaviour
IDLE	Waits for start pulse. Validates config: length > 0, addresses word-aligned. Asserts error and stays in IDLE if invalid.
READ_REQ	Issues read request to source address (m_addr=src, m_read='1'). Waits for m_ready.
WRITE_REQ	Writes buffered word to destination (m_wstrb=1111, m_write='1'). Increments src/dst by 4, decrements counter. Returns to READ_REQ or transitions to DONE.
DONE	Sets done='1', generates IRQ if enabled, releases bus (active_o='0'). Returns to IDLE.

10.4.4 Connections to Adjacent Modules

- ← bus_interconnect (slave): sel, addr, wdata, wstrb, write_en, read_en, valid
- → bus_interconnect (slave): rdata, ready, error
- → soc_top arbitration (master): m_addr, m_wdata, m_wstrb, m_write, m_read, m_valid, active_o
- ← soc_top arbitration (master): m_rdata, m_ready, m_error
- → soc_top: dma_irq (combined with timer_irq → core irq_in)

11. Reset Synchronisation and Clock Domains

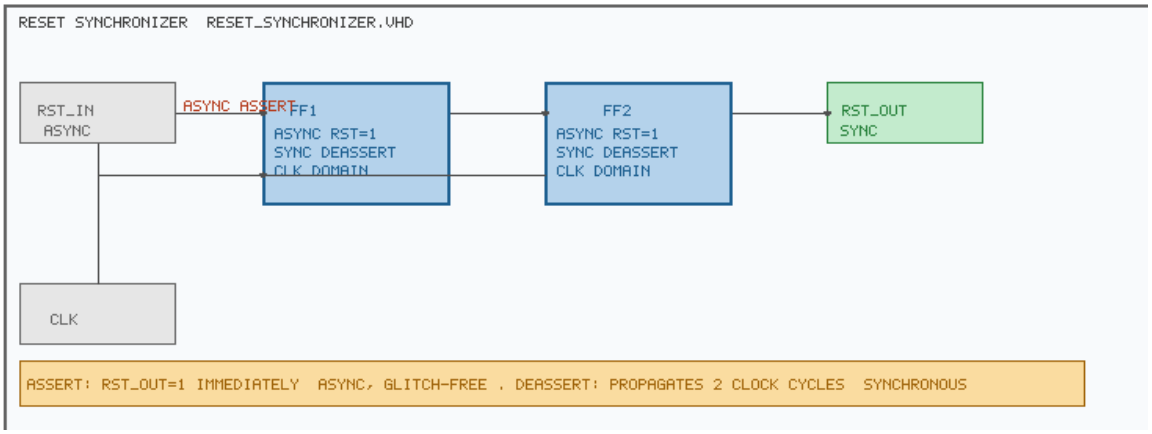

Figure 9 — reset_synchronizer.vhd: asynchronous assert, synchronous deassert

11.1 reset_synchronizer.vhd

In multi-clock-domain systems, reset must be handled carefully. The failure mode is deassert metastability: if reset is released asynchronously relative to the local clock, flip-flops may exit reset at different times, corrupting state.

The solution adopted is asymmetric reset synchronisation: assert is asynchronous (for immediate response to power-on or emergency conditions), while deassert is synchronised to the local clock.

11.1.1 Implementation

A 2-flip-flop shift register with asynchronous reset. When rst_in = '1': both FFs are immediately forced to '1'. When rst_in = '0': the '0' propagates through the 2-FF pipeline, arriving at rst_out synchronised to the local clock with exactly 2 cycles of latency.

11.1.2 Instantiations in soc_top

- rst_core_sync: clocked by clk — produces rst_core for rv32i_core, data_memory, gpio, timer, dma.
- rst_periph_sync: clocked by periph_clk — produces rst_periph for the UART TX engine.

11.2 Simulation Clock Frequencies

To exercise real CDC behaviour in simulation (preventing artificially aligned clock edges), the two clocks use incommensurate periods:

- core_clk period: 10 ns (100 MHz)
- periph_clk period: 14 ns (~71 MHz)

With these frequencies, clock edges rarely coincide, properly exercising the 2-FF synchronisers and validating the toggle handshake protocol.

12. Verification Strategy

12.1 Testbench Coverage

The project uses a two-level verification strategy: unit tests for every individual module, and an integration testbench that validates the complete system running real firmware.

Testbench	Coverage
tb_alu.vhd	All 11 ALU operations, corner cases: zero flag, shift by 0 and 31, overflow
tb_alu_control.vhd	All opcodes and funct3/funct7 combinations producing each operation code
tb_branch_compare.vhd	All 6 branch conditions, signed vs unsigned, equality
tb_control_unit.vhd	All recognised opcodes, illegal instruction, ECALL, EBREAK, FENCE
tb_immediate_generator.vhd	All 5 immediate formats, sign-extension verification
tb_load_store_unit.vhd	All 5 load types, 3 store types, all addr[1:0] offsets, misalignment detection
tb_program_counter.vhd	Reset, increment, arbitrary value load, enable
tb_register_file.vhd	Read/write all registers, x0 hardwired zero
tb_trap_unit.vhd	All trap sources, priority between simultaneous sources
tb_data_memory.vhd	Read/write with all byte-enable combinations, reset
tb_gpio.vhd	GPIO_OUT write, GPIO_IN read, invalid address error
tb_timer.vhd	Free-running counter, comparator match, IRQ generation, clear
tb_uart.vhd	8N1 transmission with distinct clock domains, CDC handshake, tx_busy
tb_dma.vhd	Register access, error on length=0, misalignment error, RAM-to-RAM copy, IRQ
tb_bus_interconnect.vhd	Address decode, response mux, unmapped address error
tb_reset_synchronizer.vhd	Asynchronous assert, synchronous deassert timing
tb_rv32i_core.vhd	Instruction sequence execution, branches, loads/stores
tb_soc_top.vhd	Full integration: firmware counter 0-10 with GPIO button control

12.2 Demonstration Firmware (program.mem)

The firmware implements a 0-to-10 (modulo 11) counter controlled by a push button via GPIO_IN. The behaviour:

- Initial state (STOP): counter is paused. Pressing the button transitions to RUN state.

- RUN state: each loop iteration increments the counter and writes the value to GPIO_OUT[3:0]. At count=10, it wraps to 0. Pressing the button transitions to STOP.
- Edge detection: the firmware detects the button's rising edge (0→1 transition) by comparing with the previous sampled value, preventing spurious toggles.

This firmware exercises the complete instruction set: fetch, arithmetic, conditional branches, subroutine calls (JAL), memory loads/stores, and GPIO read/write.

13. Roadmap — Planned Extensions

This chapter documents the planned architectural evolution, organised into prioritised incremental steps. Each step was specified in detail and can be implemented independently without breaking existing functionality.

13.1 Step 1 — Interrupt Controller

The system already generates IRQs from the timer and DMA, but has no centralised controller. The next major extension is an interrupt_controller peripheral at reserved address 0xC0:

- Registers: IRQ_PENDING (RO), IRQ_MASK (RW), IRQ_ACK (WO), IRQ_STATUS (RO).
- Sources: timer_irq, dma_irq, and future uart_irq, spi_irq.
- Integration with rv32i_core: the irq_in port already exists on the core boundary. The core needs a basic external interrupt handler: detect irq_in when enabled, suppress current instruction side-effects, and branch to an IRQ vector.
- Acceptance criterion: firmware that configures the timer, enables IRQ, and demonstrates correct ISR dispatch via the interrupt vector.

13.2 Step 2 — SPI Peripheral

Add a memory-mapped SPI master at reserved address 0x60:

- Registers: SPI_TXDATA (WO), SPI_RXDATA (RO), SPI_STATUS (RO), SPI_CTRL (RW) — already reserved in the memory map.
- Configurable CPOL/CPHA modes, programmable SCK divisor.
- Physical outputs: SCK, MOSI, MISO, CSn.

13.3 Step 3 — Asynchronous FIFO (async_fifo.vhd)

A parameterisable Gray-coded pointer FIFO for safe multi-bit data transfer between clock domains:

- Separate write port (domain A) and read port (domain B).
- Gray-coded pointers crossing the domain boundary.
- Independent full/empty status per domain.
- Planned use: data path between DMA and serial peripherals; multi-channel UART.
- Acceptance criterion: formal verification that read-from-empty and write-to-full never occur; tests with distinct asynchronous clocks.

13.4 Step 4 — CDC Control Bridge (cdc_control_bridge.vhd)

A reusable parameterisable module for crossing control signals (start, done, status, configuration handshake) between clock domains, replacing the inline handshake currently embedded in uart.vhd.

13.5 Step 5 — DMA Expansion

- RAM-to-peripheral transfers (RAM → UART_TXDATA): enables DMA-driven string transmission.
- Second DMA channel for parallel transfers.
- Byte-addressable DMA_LENGTH (instead of word-granularity).

13.6 Step 6 — 5-Stage Pipeline

The most ambitious core extension: transform the single-cycle processor into a classic 5-stage pipeline (IF / ID / EX / MEM / WB):

- Pipeline registers between stages (IF/ID, ID/EX, EX/MEM, MEM/WB flip-flops).
- Data hazard detection and forwarding paths (EX-MEM, MEM-WB forwarding).
- Load-use stall detection: pipeline stall when a load result is used immediately.
- Control hazard handling: pipeline flush on taken branches.
- The enable signal on program_counter (already present but unused) becomes the stall mechanism.

Expected benefit: significantly higher clock frequency (the critical path is shortened to a single pipeline stage), yielding much higher throughput at the same technology node.

13.7 Step 7 — Advanced Verification

- VHDL assertions: inline property checks (e.g., never bus_write='1' and bus_read='1', never two sel signals active simultaneously).
- Protocol checkers: modules that monitor bus transactions and flag violations.
- Functional coverage: metrics ensuring all relevant DMA FSM state transitions were exercised.
- Timeout checks: ensuring bus_ready never stays '0' indefinitely.

13.8 Step 8 — FPGA Demo

- Boot message over UART: on power-on, the SoC transmits a boot string visible in any serial terminal (115200 baud, 8N1).
- Interactive demo: GPIO buttons control the counter; LEDs display the count.
- DMA benchmark: copy a memory block with timer measurement, comparing CPU-driven vs DMA-driven performance.

- XDC constraints: Basys3 pin mapping for buttons, LEDs, and UART TX pad; timing constraints for both clock domains.

13.9 Priority and Dependency Matrix

Step	Priority	Difficulty	Requires
1. Interrupt Controller	High	Medium	Stable SoC v1
2. SPI Peripheral	Medium	Medium	Stable SoC v1
3. Async FIFO	Medium	High	Active CDC domains
4. CDC Control Bridge	Medium	High	Step 3
5. DMA Expansion	Medium	High	Steps 1, 3
6. 5-Stage Pipeline	Long-term	Very high	Stable SoC
7. Advanced Verification	Ongoing	High	Any step
8. FPGA Demo	High	Medium	Steps 1, 2

14. References

14.1 RISC-V Specifications

- RISC-V International. The RISC-V Instruction Set Manual, Vol. I: Unprivileged ISA. Version 20191213. riscv.org
- RISC-V International. The RISC-V Instruction Set Manual, Vol. II: Privileged Architecture. Version 20211203.

14.2 VHDL and Digital Design

- IEEE Std 1076-1993 / 1076-2002. IEEE Standard VHDL Language Reference Manual.
- Ashenden, P. J. The Designer's Guide to VHDL. 3rd ed. Morgan Kaufmann, 2008.
- Pong P. Chu. FPGA Prototyping by VHDL Examples. Wiley, 2008.

14.3 Computer Architecture

- Patterson, D. A.; Hennessy, J. L. Computer Organization and Design RISC-V Edition. 2nd ed. Morgan Kaufmann, 2020.
- Harris, D.; Harris, S. Digital Design and Computer Architecture: RISC-V Edition. Morgan Kaufmann, 2021.

14.4 SoC Design and Protocols

- ARM Limited. AMBA APB Protocol Specification v2.0. ARM IHI 0024C, 2010.
- Cummings, C. E. Synthesis and Scripting Techniques for Designing Multi-Asynchronous Clock Designs. SNUG 2001.
- Cummings, C. E.; Alfke, P. Simulation and Synthesis Techniques for Asynchronous FIFO Design. SNUG 2002.

Mini-SoC RV32I in VHDL — Complete Technical Documentation

Mateus Telles Nebias